Name - Peiris.T.S.R

Index no -230470U

repo https://github.com/angstrom10/EN3160---Image-Processing-and-Machine-Vision

Imports

In [128…
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_laplace, maximum_filter
from pathlib import Path
import csv
from scipy.optimize import minimize
```

Q1

In [129…
```
im = cv2.imread("C:\\EN3160 - Image Processing and Machine Vision\\a-02\\a2images\\a2images\\the_berry_farms_sunflower_field.j
gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
gray = gray.astype(np.float32) / 255.0
print("Image dimensions:", gray.shape)
```

```
Image dimensions: (360, 360)
```

In [130…
```
SIGMAS = np.linspace(1.0, 12.0, num=25)
RELATIVE_THRESHOLD = 0.6
```

In [131…
```
responses = []
for sigma in SIGMAS:
    log_response = gaussian_laplace(gray, sigma=sigma)
    normalized_log = (sigma ** 2) * log_response
    response = np.abs(normalized_log)
    responses.append(response)
```

In [132…
```
scale_space = np.stack(responses, axis=0)
local_max = maximum_filter( scale_space,size=(3, 5,5),mode="nearest")
threshold = RELATIVE_THRESHOLD * float(scale_space.max())
is_extremum = ((scale_space == local_max)& (scale_space >= threshold))
scale_idx, y_idx, x_idx = np.where(is_extremum)

blobs = []
for si, y, x in zip(scale_idx, y_idx, x_idx):
    sigma = float(SIGMAS[si])
    response = float(scale_space[si, y, x])
    radius = np.sqrt(2.0) * sigma
    blobs.append(
        (response, int(x), int(y), sigma, float(radius))
    )
blobs.sort(key=lambda item: item[0], reverse=True)
```

In [133…
```
result = im.copy()

for response, x, y, sigma, radius in blobs:
    cv2.circle(
        result,
        (x, y),
        max(1, int(round(radius))),
        (0, 0, 255),
        1,
        cv2.LINE_AA
    )

largest = sorted(
    blobs,
    key=lambda item: item[4],
    reverse=True
)[:10]

print("\nLargest detected circles")
print("Rank | x | y | sigma | radius | LoG response")
```

```python
for rank, (response, x, y, sigma, radius) in enumerate(
    largest, start=1
):
    print(
        f"{rank:4d} | {x:4d} | {y:4d} | "
        f"{sigma:5.2f} | {radius:6.2f} | "
        f"{response:.5f}"
    )

with open("q1_largest_circles.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(
        ["rank", "x", "y", "sigma", "radius_pixels", "log_response"]
    )

    for rank, blob in enumerate(largest, start=1):
        response, x, y, sigma, radius = blob
        writer.writerow(
            [rank, x, y, sigma, radius, response]
        )


result_rgb = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)
original_rgb = cv2.cvtColor(im, cv2.COLOR_BGR2RGB)

fig, axes = plt.subplots(1, 2, figsize=(15, 7))

axes[0].imshow(original_rgb)
axes[0].set_title("Original sunflower image")
axes[0].axis("off")

axes[1].imshow(result_rgb)
axes[1].set_title("Scale-space LoG blob detections")
axes[1].axis("off")

plt.tight_layout()
plt.savefig("q1_log_detection_result.png", dpi=200)
plt.show()

max_response_image = scale_space.max(axis=0)

# plt.figure(figsize=(9, 7))
# plt.imshow(max_response_image, cmap="inferno")
# plt.colorbar(label="Maximum normalized |LoG| response")
# plt.title("Maximum LoG response across scales")
# plt.axis("off")
# plt.tight_layout()
# plt.savefig("q1_log_response.png", dpi=200)
# plt.show()
```

```
Largest detected circles
Rank | x | y | sigma | radius | LoG response
   1 |  293 |  169 | 12.00 |  16.97 | 0.24615
   2 |  337 |  164 | 12.00 |  16.97 | 0.24440
   3 |  282 |  338 |  7.42 |  10.49 | 0.34865
   4 |    0 |  275 |  7.42 |  10.49 | 0.33942
   5 |  223 |  174 |  6.50 |   9.19 | 0.25183
   6 |  106 |  255 |  6.04 |   8.54 | 0.36307
   7 |  179 |  327 |  6.04 |   8.54 | 0.34574
   8 |  178 |  261 |  5.58 |   7.90 | 0.35272
   9 |   53 |  174 |  5.58 |   7.90 | 0.30172
  10 |  187 |  341 |  5.58 |   7.90 | 0.26269
```

Original sunflower image



Scale-space LoG blob detections



Q2

In [134…

```
np.random.seed(0)

N = 100
half_n = N // 2

# Ground-truth circle
r_gt = 10
x0_gt, y0_gt = 2, 3
s_circle = r_gt / 16

t = np.random.uniform(0, 2 * np.pi, half_n)
n = s_circle * np.random.randn(half_n)

x = x0_gt + (r_gt + n) * np.cos(t)
y = y0_gt + (r_gt + n) * np.sin(t)
X_circ = np.column_stack((x, y))

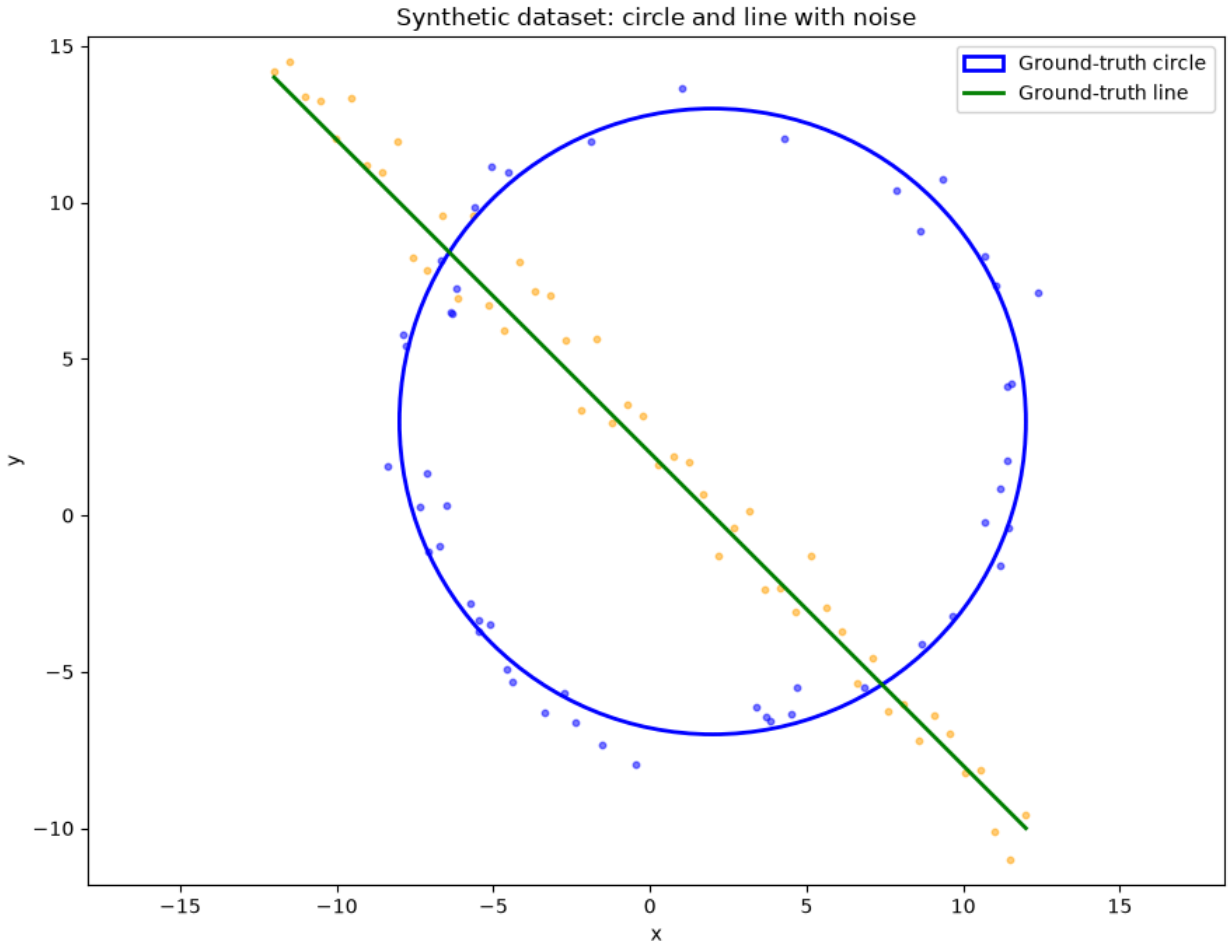
# Ground-truth line: y = mx + b
m_gt, b_gt = -1, 2
s_line = 1.0

x = np.linspace(-12, 12, half_n)
y = m_gt * x + b_gt + s_line * np.random.randn(half_n)
X_line = np.column_stack((x, y))

# Combined point set
X = np.vstack((X_circ, X_line))
```

In [135…

```
plt.figure(figsize=(9, 7))
plt.scatter(X_line[:, 0], X_line[:, 1], s=10, color="orange", alpha=0.5)
plt.scatter(X_circ[:, 0], X_circ[:, 1], s=10, color="blue", alpha=0.5)
circle = plt.Circle((x0_gt, y0_gt), r_gt, color="blue", fill=False, lw=2, label="Ground-truth circle")
plt.gca().add_artist(circle)
line_x = np.array([-12, 12])
line_y = m_gt * line_x + b_gt
plt.plot(line_x, line_y, color="green", lw=2, label="Ground-truth line")
plt.title("Synthetic dataset: circle and line with noise")
plt.xlabel("x")
plt.ylabel("y")
plt.axis("equal")
plt.tight_layout()
plt.legend()
plt.show()
```



RANSAC LINE FITTING

In [136…

```python
def line_from_two_points(p1, p2):
    direction = p2 - p1
    length = np.linalg.norm(direction)
    if length < 1e-10:
        return None
    normal = np.array([-direction[1], direction[0]]) / length
    d = np.dot(normal, p1)
    return normal[0], normal[1], d

def line_distances(points, model):
    a, b, d = model
    return np.abs(a * points[:, 0] + b * points[:, 1] - d)

def ransac_line(points, iterations=3000, threshold=1.8,
                min_inliers=35):
    best_model = None
    best_inliers = None
    best_sample = None

    for _ in range(iterations):
        # A line requires two sample points
        indices = np.random.choice(len(points), 2, replace=False)
        sample = points[indices]

        model = line_from_two_points(sample[0], sample[1])

        if model is None:
            continue

        distances = line_distances(points, model)
        inliers = distances < threshold

        if inliers.sum() >= min_inliers:
            if (best_inliers is None or
                    inliers.sum() > best_inliers.sum()):
```

```
                best_model = model
                best_inliers = inliers
                best_sample = sample.copy()

    return best_model, best_inliers, best_sample
line_fit, line_inliers, line_sample = ransac_line(X)
print("Best line model (a, b, d):", line_fit)
print("Number of inliers:", line_inliers.sum())
```

```
Best line model (a, b, d): (np.float64(0.7060506323381652), np.float64(0.7081613549007578), np.float64(1.6919486446248255))
Number of inliers: 57
```

In [137…

```
plt.figure(figsize=(9, 7))

plt.scatter(X_line[:, 0], X_line[:, 1],
            s=10, color="orange", alpha=0.5, label="Line points")
plt.scatter(X_circ[:, 0], X_circ[:, 1],
            s=10, color="blue", alpha=0.5, label="Circle points")

plt.scatter(X[line_inliers, 0], X[line_inliers, 1],
            s=25, color="olive", label="Line inliers")

plt.scatter(line_sample[:, 0], line_sample[:, 1],
            s=100, marker="x", color="red",
            label="Best two-point sample")

circle = plt.Circle((x0_gt, y0_gt), r_gt,
                    color="blue", fill=False, lw=2,
                    label="Ground-truth circle")
plt.gca().add_artist(circle)

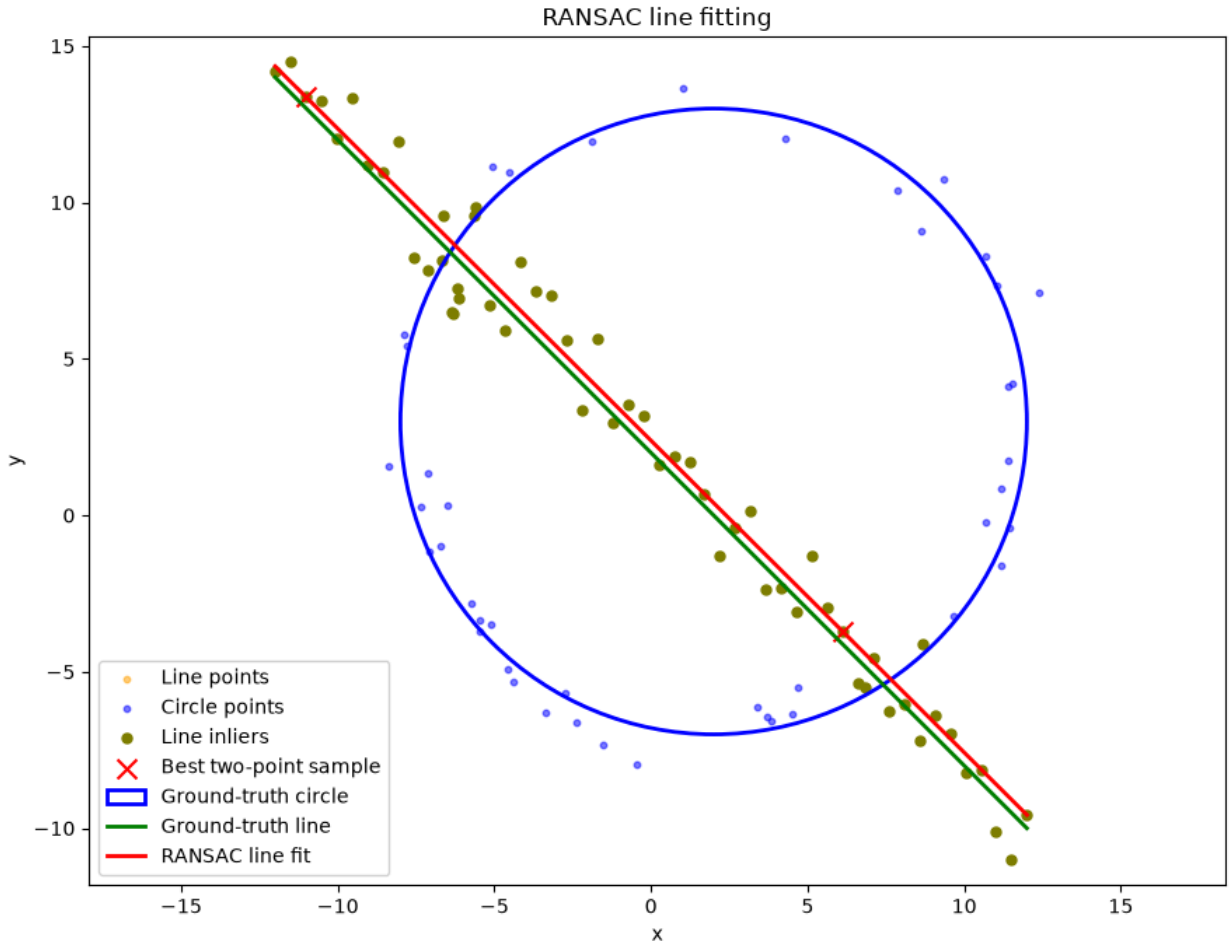
line_x = np.array([-12, 12])
line_y = m_gt * line_x + b_gt
plt.plot(line_x, line_y, color="green", lw=2,
         label="Ground-truth line")

# Convert ax + by = d to y = mx + c

a, b, d = line_fit

if abs(b) > 1e-10:
    line_y_ransac = (d - a * line_x) / b
    plt.plot(line_x, line_y_ransac, color="red", lw=2,
             label="RANSAC line fit")
else:
    plt.axvline(d / a, color="red", lw=2,
                label="RANSAC line fit")

plt.title("RANSAC line fitting")
plt.xlabel("x")
plt.ylabel("y")
plt.axis("equal")
plt.tight_layout()
plt.legend()
plt.show()
```



In [138…

```python
def fit_line_tls(points):
    centroid = points.mean(axis=0)
    centered = points - centroid

    # SVD: smallest singular direction is the line normal
    _, _, vt = np.linalg.svd(centered, full_matrices=False)

    normal = vt[-1]
    normal = normal / np.linalg.norm(normal)
    d = np.dot(normal, centroid)

    return normal[0], normal[1], d


# Refine line using the RANSAC consensus
line_best = fit_line_tls(X[line_inliers])

# Remove line inliers for Question 2(b)
X_remaining = X[~line_inliers]

print("QUESTION 2(a): LINE FITTING")
print("RANSAC line (a, b, d):", line_fit)
print("Refined TLS line (a, b, d):", line_best)
print("Unit-normal magnitude:",
      np.linalg.norm(line_best[:2]))
print("Number of line inliers:", line_inliers.sum())
print("Points remaining:", len(X_remaining))
```

```
QUESTION 2(a): LINE FITTING
RANSAC line (a, b, d): (np.float64(0.7060506323381652), np.float64(0.7081613549007578), np.float64(1.6919486446248255))
Refined TLS line (a, b, d): (np.float64(0.7156488270044808), np.float64(0.6984602754681979), np.float64(1.5617747756951443))
Unit-normal magnitude: 1.0
Number of line inliers: 57
Points remaining: 43
```

CIRCLE ESTIMATION FROM THREE POINTS

In [139…

```python
def circle_from_three_points(points):

    p1, p2, p3 = points

    A = 2 * np.array([
        [p2[0] - p1[0], p2[1] - p1[1]],
        [p3[0] - p1[0], p3[1] - p1[1]]
    ])

    rhs = np.array([
        p2 @ p2 - p1 @ p1,
        p3 @ p3 - p1 @ p1
    ])

    if abs(np.linalg.det(A)) < 1e-8:
        return None

    cx, cy = np.linalg.solve(A, rhs)
    radius = np.linalg.norm(p1 - np.array([cx, cy]))

    if not np.isfinite(radius):
        return None

    return cx, cy, radius


def circle_distances(points, model):

    cx, cy, radius = model
    distances = np.linalg.norm(
        points - np.array([cx, cy]), axis=1
    )
    return np.abs(distances - radius)
```

RANSAC CIRCLE FITTING

In [140…

```python
def ransac_circle(points, iterations=5000, threshold=1.2,
                  min_inliers=30):
    best_model = None
    best_inliers = None
    best_sample = None

    if len(points) < 3:
        raise RuntimeError("Not enough points to fit a circle.")

    for _ in range(iterations):
        # Three non-collinear points determine a circle
        indices = np.random.choice(len(points), 3, replace=False)
        sample = points[indices]
        model = circle_from_three_points(sample)
        if model is None:
            continue
        distances = circle_distances(points, model)
        inliers = distances < threshold
        if inliers.sum() >= min_inliers:
            if (best_inliers is None or
                    inliers.sum() > best_inliers.sum()):
                best_model = model
                best_inliers = inliers
                best_sample = sample.copy()

    if best_model is None:
        raise RuntimeError(
            "No acceptable circle consensus found. "
            "Adjust the threshold or minimum consensus."
        )

    return best_model, best_inliers, best_sample


circle_ransac, circle_inliers, circle_sample = ransac_circle(
    X_remaining
)
```

In [141…

```python
fig, ax = plt.subplots(figsize=(9, 7))

# Original line and circle points
ax.scatter(
```

```
    X_line[:, 0], X_line[:, 1],
    s=10, color="orange", alpha=0.5,
    label="Line points"
)

ax.scatter(
    X_circ[:, 0], X_circ[:, 1],
    s=10, color="blue", alpha=0.5,
    label="Circle points"
)

# Ground-truth circle
gt_circle = plt.Circle(
    (x0_gt, y0_gt), r_gt,
    color="blue", fill=False, linewidth=2,
    label="Ground-truth circle"
)
ax.add_patch(gt_circle)

# Ground-truth line
line_x = np.array([-12, 12])
line_y = m_gt * line_x + b_gt

ax.plot(
    line_x, line_y,
    color="green", linewidth=2,
    label="Ground-truth line"
)

# Remaining points after line fitting
ax.scatter(
    X_remaining[:, 0], X_remaining[:, 1],
    c="gray", s=20, alpha=0.6,
    label="Remaining points"
)

# RANSAC circle
cx, cy, radius = circle_ransac

ransac_circle_patch = plt.Circle(
    (cx, cy), radius,
    color="red", fill=False, linewidth=2,
    label="RANSAC circle fit"
)
ax.add_patch(ransac_circle_patch)

# Circle inliers
ax.scatter(
    X_remaining[circle_inliers, 0],
    X_remaining[circle_inliers, 1],
    c="limegreen", s=25,
    label="Circle inliers"
)

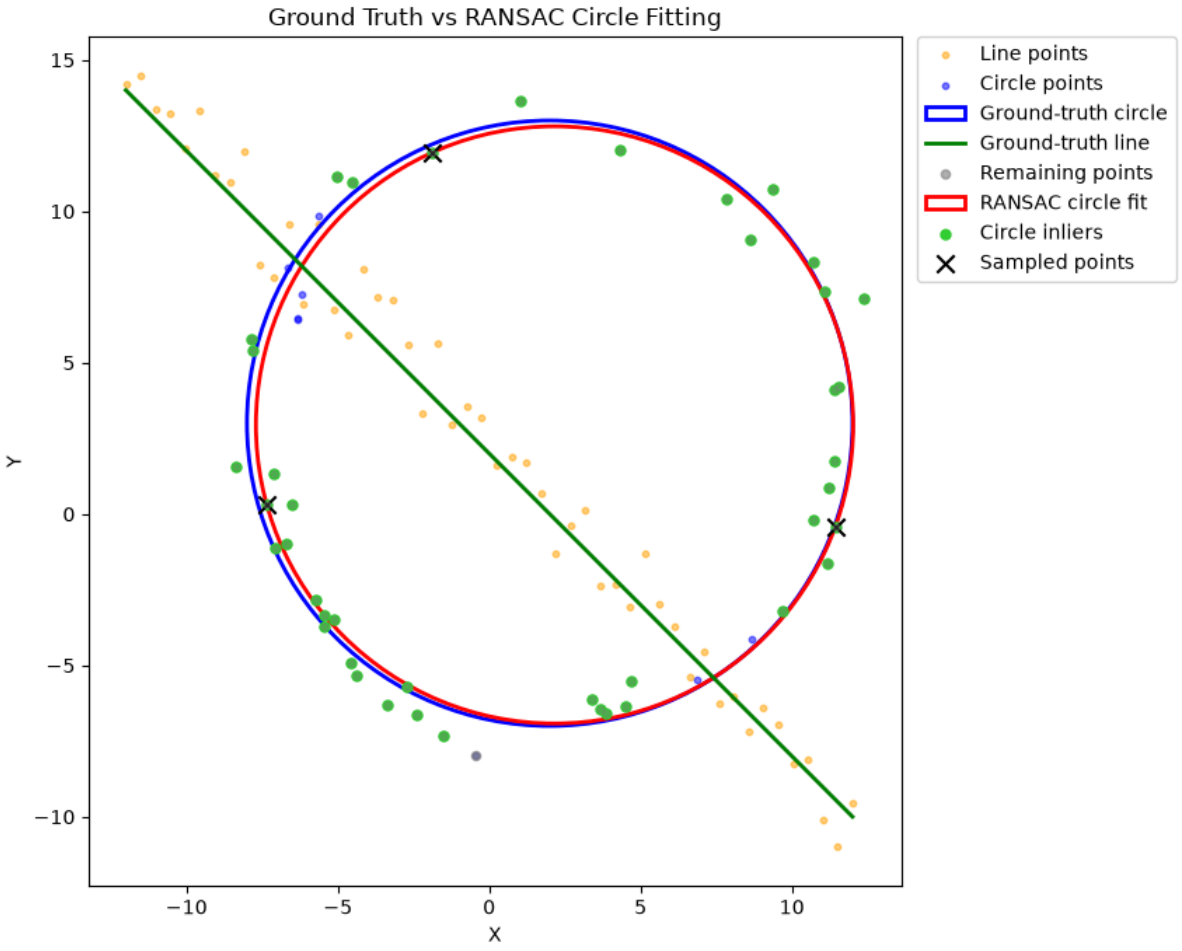
# Three points used by the best RANSAC sample
ax.scatter(
    circle_sample[:, 0], circle_sample[:, 1],
    c="black", marker="x", s=80,
    label="Sampled points"
)

ax.set_aspect("equal", adjustable="box")
ax.set_title("Ground Truth vs RANSAC Circle Fitting")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.legend(
    loc="upper left",
    bbox_to_anchor=(1.02, 1),
    borderaxespad=0
)

ax.scatter(
    X_remaining[:, 0],
    X_remaining[:, 1],
    c="gray",
    s=12,
    alpha=0.35,
    label="_nolegend_"
)
```

```
plt.tight_layout()
plt.show()
ax.grid(alpha=0.2)


print("RANSAC circle (cx, cy, radius):", circle_ransac)
print("Number of circle inliers:", circle_inliers.sum())
print("Number of remaining points:", len(X_remaining))
```




```
RANSAC circle (cx, cy, radius): (np.float64(2.1590326761762193), np.float64(2.9438434924786905), np.float64(9.863815359872813))
Number of circle inliers: 42
Number of remaining points: 43
```

In [142…

```
from scipy.optimize import minimize

def circle_objective(params, points):
    cx, cy, radius = params

    if radius <= 0:
        return 1e12

    distances = np.linalg.norm(
        points - np.array([cx, cy]), axis=1
    )

    errors = distances - radius

    return np.sum(errors ** 2)


# Initial estimate from RANSAC
initial_circle = np.array(circle_ransac, dtype=float)

# Optimize using circle inliers
result = minimize(
    circle_objective,
    initial_circle,
    args=(X_remaining[circle_inliers],),
    method="Nelder-Mead",
    options={
```

```
        "maxiter": 10000,
        "xatol": 1e-9,
        "fatol": 1e-9
    }
)

if not result.success:
    print("Optimization warning:", result.message)

circle_best = result.x

print("QUESTION 2(b): CIRCLE FITTING")
print("RANSAC circle (cx, cy, radius):", circle_ransac)
print("Refined circle (cx, cy, radius):", circle_best)
print("Ground-truth circle:", (x0_gt, y0_gt, r_gt))
print("Number of circle inliers:", circle_inliers.sum())
```

```
QUESTION 2(b): CIRCLE FITTING
RANSAC circle (cx, cy, radius): (np.float64(2.1590326761762193), np.float64(2.9438434924786905), np.float64(9.863815359872813))
Refined circle (cx, cy, radius): [1.86852053 3.12615243 9.92984015]
Ground-truth circle: (2, 3, 10)
Number of circle inliers: 42
```

In [143…

```
plt.figure(figsize=(10, 9))

# All original points
plt.scatter(X[:, 0], X[:, 1], s=15, color="lightgray",
            alpha=0.6, label="All points")

# Line inliers
plt.scatter(X[line_inliers, 0], X[line_inliers, 1],
            s=25, color="olive", label="Line inliers")

# Circle inliers
circle_points = X_remaining[circle_inliers]
plt.scatter(circle_points[:, 0], circle_points[:, 1],
            s=25, color="cyan", label="Circle inliers")

# Best samples
plt.scatter(line_sample[:, 0], line_sample[:, 1],
            marker="x", s=120, color="red",
            label="Best line sample")

plt.scatter(circle_sample[:, 0], circle_sample[:, 1],
            marker="*", s=160, color="darkred",
            label="Best circle sample")

# Ground-truth circle
theta = np.linspace(0, 2 * np.pi, 500)

plt.plot(
    x0_gt + r_gt * np.cos(theta),
    y0_gt + r_gt * np.sin(theta),
     linewidth=2, label="Ground-truth circle"
)

# RANSAC circle
cx, cy, radius = circle_ransac
plt.plot(
    cx + radius * np.cos(theta),
    cy + radius * np.sin(theta),
    color="magenta", linestyle=":", linewidth=2,
    label="RANSAC circle"
)


# Ground-truth line
line_x = np.array([-15, 15])
plt.plot(line_x, m_gt * line_x + b_gt,
          linewidth=2, label="Ground-truth line")

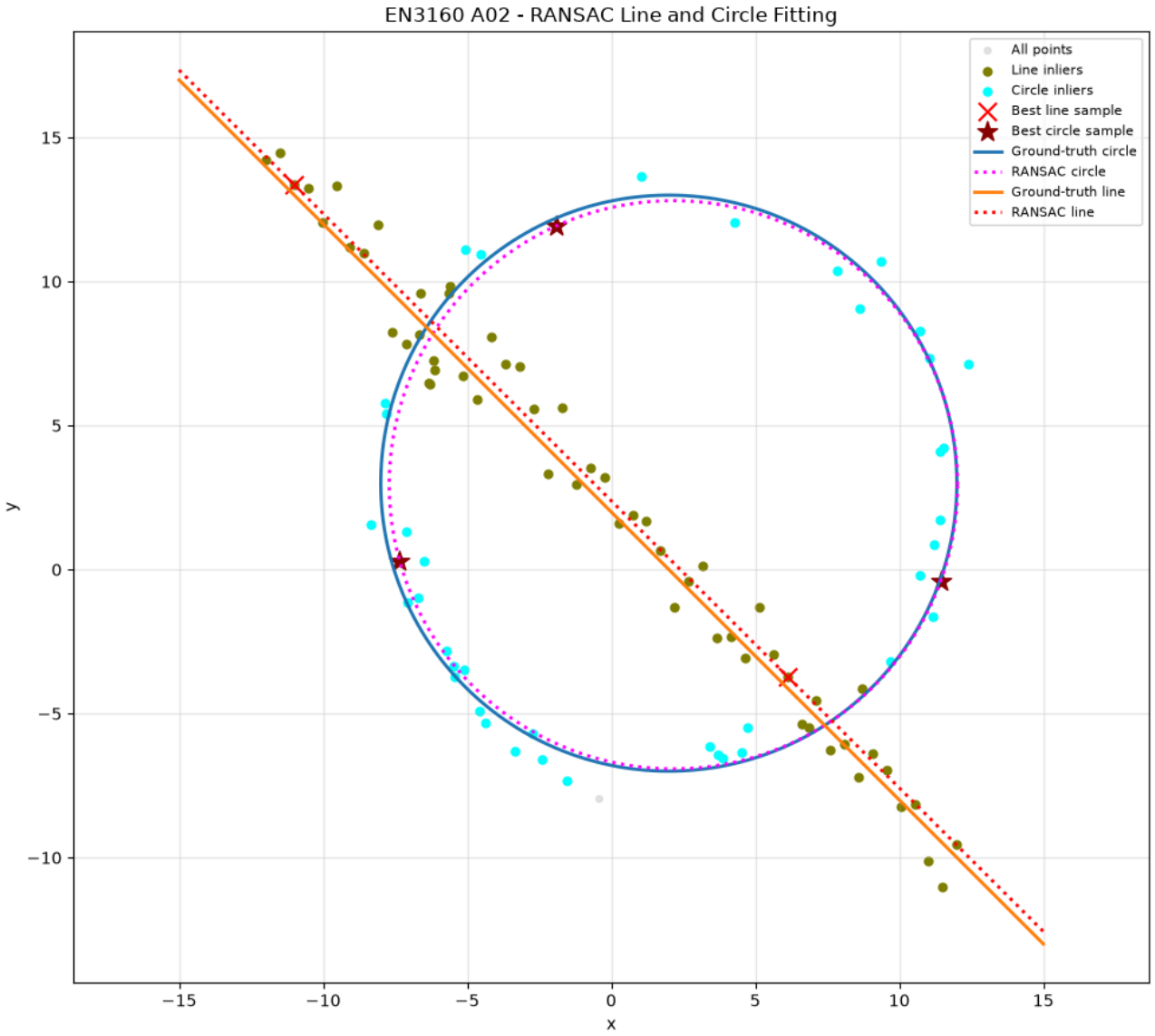
# RANSAC line
a, b, d = line_fit
if abs(b) > 1e-10:
    plt.plot(line_x, (d - a * line_x) / b,
             color="red", linestyle=":", linewidth=2,
             label="RANSAC line")



plt.title("EN3160 A02 - RANSAC Line and Circle Fitting")
```

```
plt.xlabel("x")
plt.ylabel("y")
plt.axis("equal")
plt.grid(alpha=0.3)
plt.legend(fontsize=8)
plt.tight_layout()

plt.savefig("q2_ransac_results.png", dpi=300)
plt.show()
```



Question 2(d): What happens if we fit the circle first?

If we fit the circle first, RANSAC can identify the circle by selecting three points and finding the model with the largest consensus. However, some points belonging to the line may also fall within the circle's radial error threshold and be incorrectly classified as circle inliers.

After removing the circle inliers, the remaining points can be used to estimate the line using RANSAC. However, if too many line points are incorrectly classified as circle inliers, the line estimation may become less accurate or fail to obtain the required consensus.

Therefore, the order of fitting can affect the results because the RANSAC thresholds and consensus sets determine which points are assigned to each model.

Q3

In [144…

```
import cv2
import matplotlib.pyplot as plt
from pathlib import Path
```

```
# Image paths
background_path = Path( r"C:\EN3160 - Image Processing and Machine Vision\a-02\a2images\a2images\wadham.jpg")
flag_path = Path(r"C:\EN3160 - Image Processing and Machine Vision\a-02\a2images\a2images\flag.png")

background = cv2.imread(str(background_path))
flag = cv2.imread(str(flag_path))
background_rgb = cv2.cvtColor(background, cv2.COLOR_BGR2RGB)
flag_rgb = cv2.cvtColor(flag, cv2.COLOR_BGR2RGB)
```

In [145…

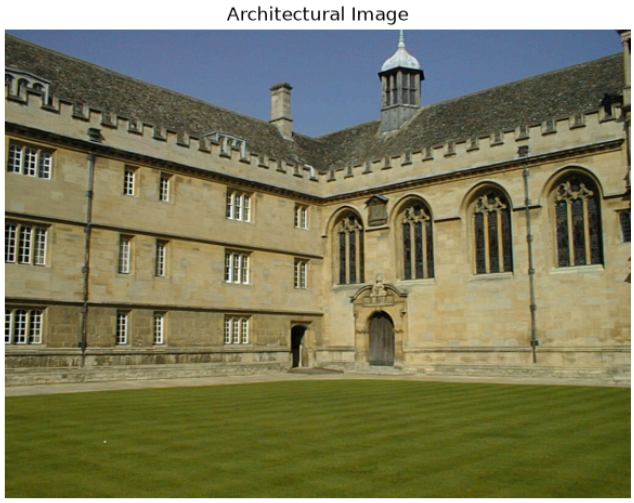
```
# Display images
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].imshow(background_rgb)
axes[0].set_title("Architectural Image")
axes[0].axis("off")

axes[1].imshow(flag_rgb)
axes[1].set_title("Flag Image")
axes[1].axis("off")

plt.tight_layout()
plt.show()

print("Background dimensions:", background.shape)
print("Flag dimensions:", flag.shape)
```






```
Background dimensions: (768, 1024, 3)
Flag dimensions: (960, 1920, 3)
```

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt

# Display the architectural image
plt.figure(figsize=(12, 8))
plt.imshow(background_rgb)
plt.title("Select 4 corners: TL, TR, BR, BL")
plt.axis("image")
plt.show()
# Click four points on the planar surface
points = plt.ginput(4, timeout=0)
plt.close()

# Store selected coordinates
destination_points = np.array(points, dtype=np.float32)

if len(destination_points) != 4:
    raise ValueError("You must select exactly four points.")

print("Selected destination points:")
print(destination_points)

# Display and label the selected corners
plt.figure(figsize=(12, 8))
plt.imshow(background_rgb)

labels = ["TL", "TR", "BR", "BL"]

plt.scatter(
```

```
    destination_points[:, 0],
    destination_points[:, 1],
    color="red",
    s=80
)

for label, (x, y) in zip(labels, destination_points):
    plt.text(x, y, label, color="yellow", fontsize=12)

plt.title("Selected building surface")
plt.axis("image")
plt.show()
```

In [153…

```
# Get flag dimensions
flag_h, flag_w = flag.shape[:2]

# Define the four corners of the original flag
source_points = np.array([
    [0, 0],                       # Top-left
    [flag_w - 1, 0],              # Top-right
    [flag_w - 1, flag_h - 1],     # Bottom-right
    [0, flag_h - 1]               # Bottom-left
], dtype=np.float32)

# Calculate the homography matrix
H, status = cv2.findHomography(
    source_points,
    destination_points,
    method=0
)

print("Homography matrix:")
print(H)

# Warp the flag to the architectural image dimensions
background_h, background_w = background.shape[:2]

warped_flag = cv2.warpPerspective(
    flag,
    H,
    (background_w, background_h)
)

# Display the warped flag
```

```
Homography matrix:
[[ 2.93626169e-01 -2.40404753e-02  1.49941559e+02]
 [ 1.01001572e-01  2.94817466e-01  2.07707794e+02]
 [ 2.00425530e-04 -5.33877997e-05  1.00000000e+00]]
```

In [ ]:

```
# Create a mask of the warped flag
gray_warped = cv2.cvtColor(warped_flag, cv2.COLOR_BGR2GRAY)
mask = gray_warped > 0

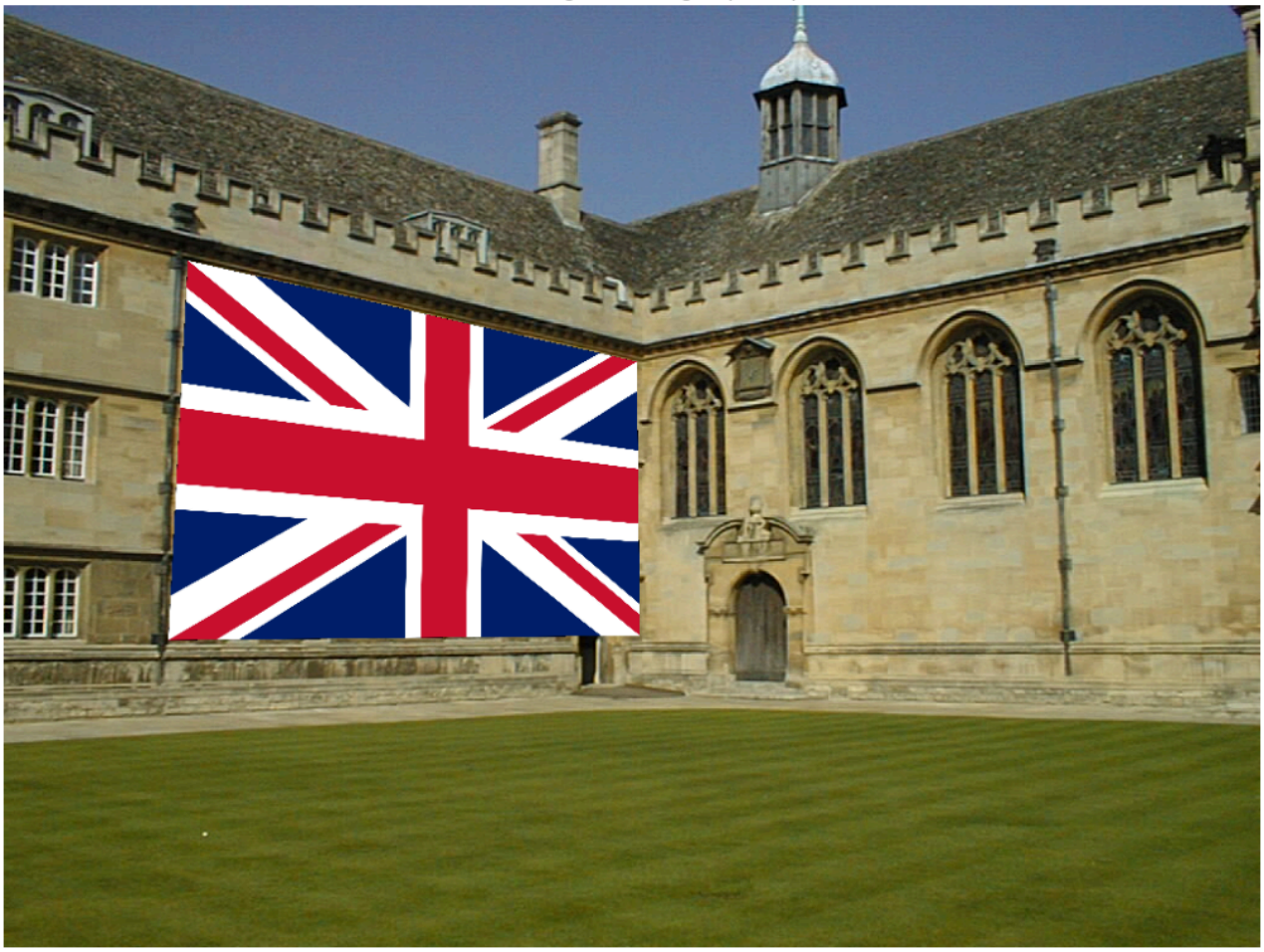
# Copy the original background
result = background.copy()

# Place the warped flag onto the background
result[mask] = warped_flag[mask]

# Convert result to RGB for display
result_rgb = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)

# Display final result
plt.figure(figsize=(12, 8))
plt.imshow(result_rgb)
plt.title("Wadham College with Flag Superimposed")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Wadham College with Flag Superimposed



In [154…
```
# Load the second architectural image
background2_path = r"C:\EN3160 - Image Processing and Machine Vision\a-02\a2images\a2images\pg.jpeg"

background2 = cv2.imread(str(background2_path))

if background2 is None:
    raise FileNotFoundError(f"Image not found: {background2_path}")

background2_rgb = cv2.cvtColor(background2, cv2.COLOR_BGR2RGB)
```

In [ ]:
```
# Select the four corners on the second image
fig, ax = plt.subplots(figsize=(12, 8))
ax.imshow(background2_rgb)
ax.set_title("Click 4 corners: TL, TR, BR, BL")
ax.axis("image")

points2 = plt.ginput(4, timeout=0, show_clicks=True)

plt.close(fig)

if len(points2) != 4:
    raise ValueError("Please select exactly four points.")

destination_points2 = np.array(points2, dtype=np.float32)

print("Second image destination points:")
print(destination_points2)
```

```
Second image destination points:
[[124.17532 240.53896]
 [602.0974  319.5    ]
 [597.9415  510.66882]
 [132.48701 477.4221 ]]
```

In [147…
```
# Load the logo
logo_path = r"C:\EN3160 - Image Processing and Machine Vision\a-02\a2images\a2images\entclogo.png"
logo = cv2.imread(str(logo_path), cv2.IMREAD_UNCHANGED)
```

```
if logo is None:
    raise FileNotFoundError(f"Logo not found: {logo_path}")

# Convert grayscale logo to BGR if necessary
if logo.ndim == 2:
    logo = cv2.cvtColor(logo, cv2.COLOR_GRAY2BGR)

# Separate transparency if the PNG has an alpha channel
if logo.shape[2] == 4:
    logo_bgr = logo[:, :, :3]
    logo_alpha = logo[:, :, 3]
else:
    logo_bgr = logo
    logo_alpha = np.full(logo.shape[:2], 255, dtype=np.uint8)

# Source corners of the logo
logo_h, logo_w = logo_bgr.shape[:2]

source_points = np.array([
    [0, 0],
    [logo_w - 1, 0],
    [logo_w - 1, logo_h - 1],
    [0, logo_h - 1]
], dtype=np.float32)

# Compute homography for the second architectural image
H2, status2 = cv2.findHomography(
    source_points, destination_points2, method=0
)

if H2 is None:
    raise RuntimeError("Could not calculate the homography.")

# Warp logo and its alpha mask
h2, w2 = background2.shape[:2]

warped_logo = cv2.warpPerspective(
    logo_bgr, H2, (w2, h2)
)

warped_alpha = cv2.warpPerspective(
    logo_alpha, H2, (w2, h2)
)

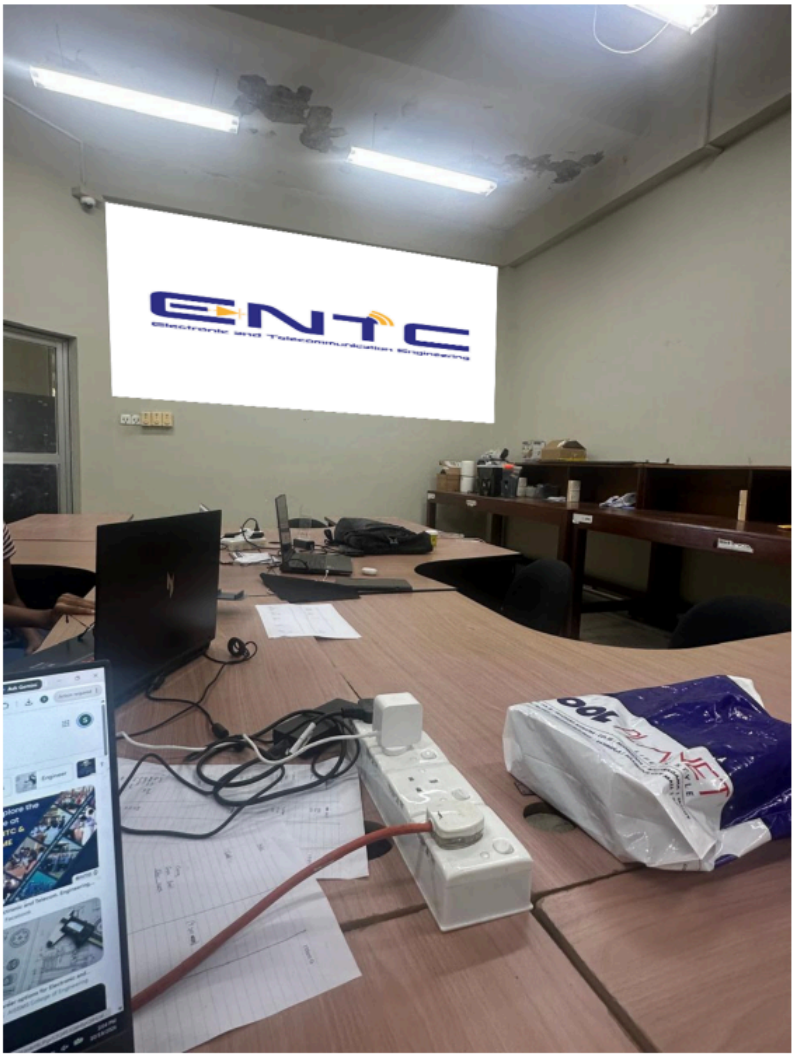
# Blend using the warped alpha mask
alpha = warped_alpha.astype(np.float32) / 255.0
alpha = alpha[:, :, np.newaxis]

result2 = (
    alpha * warped_logo.astype(np.float32)
    + (1 - alpha) * background2.astype(np.float32)
).clip(0, 255).astype(np.uint8)

# Display result inline
result2_rgb = cv2.cvtColor(result2, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(12, 8))
plt.imshow(result2_rgb)
plt.title("Second Image with Logo Superimposed")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Second Image with Logo Superimposed



Q4

In [148…

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

# Image and output folders
IMAGE_DIR = Path("C:\\EN3160 - Image Processing and Machine Vision\\a-02\\a2images\\a2images\\pcbs")
OUTPUT_DIR = Path("q4_outputs")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# Load reference template
reference_path = IMAGE_DIR / "AECC1.jpg"
template = cv2.imread(str(reference_path))

image_files = sorted([
    p for p in IMAGE_DIR.iterdir()
    if p.is_file()
    and p.suffix.lower() in (".jpg", ".jpeg", ".png")
    and p.name.lower() != "aecc1.jpg"
])

print("Template:", reference_path)
print("PCB images:", [p.name for p in image_files])

if not image_files:
    raise FileNotFoundError("No PCB comparison images found.")

# SIFT detector and descriptor matcher
sift = cv2.SIFT_create()
```

```python
bf = cv2.BFMatcher(cv2.NORM_L2)

def align_and_compare(template, pcb):
    # Convert images to grayscale
    gray_ref = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
    gray_pcb = cv2.cvtColor(pcb, cv2.COLOR_BGR2GRAY)

    # Detect SIFT features
    kp_ref, des_ref = sift.detectAndCompute(gray_ref, None)
    kp_pcb, des_pcb = sift.detectAndCompute(gray_pcb, None)

    if des_ref is None or des_pcb is None:
        raise ValueError("Could not detect SIFT descriptors.")

    # Match features using KNN and Lowe's ratio test
    pairs = bf.knnMatch(des_pcb, des_ref, k=2)

    good = [
        m for pair in pairs if len(pair) == 2
        for m, n in [pair]
        if m.distance < 0.75 * n.distance
    ]

    if len(good) < 4:
        raise ValueError("Not enough good matches for homography.")

    # Corresponding points: PCB -> reference
    src = np.float32([
        kp_pcb[m.queryIdx].pt for m in good
    ]).reshape(-1, 1, 2)

    dst = np.float32([
        kp_ref[m.trainIdx].pt for m in good
    ]).reshape(-1, 1, 2)

    # Estimate homography with RANSAC
    H, mask = cv2.findHomography(
        src, dst, cv2.RANSAC, 5.0
    )

    if H is None or mask is None:
        raise ValueError("RANSAC could not estimate homography.")

    inliers = mask.ravel().astype(bool)

    # Draw only RANSAC inlier matches
    matches_img = cv2.drawMatches(
        pcb, kp_pcb,
        template, kp_ref,
        [m for m, keep in zip(good, inliers) if keep],
        None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    # Warp PCB into reference coordinates
    height, width = gray_ref.shape

    aligned = cv2.warpPerspective(
        pcb, H, (width, height)
    )

    # Warp a validity mask to exclude artificial black borders
    source_mask = np.full(gray_pcb.shape, 255, dtype=np.uint8)

    valid = cv2.warpPerspective(
        source_mask, H, (width, height),
        flags=cv2.INTER_NEAREST
    ) > 0

    aligned_gray = cv2.cvtColor(
        aligned, cv2.COLOR_BGR2GRAY
    )

    # Absolute difference
    difference = cv2.absdiff(gray_ref, aligned_gray)

    # Calculate MAE only where the warped image has valid pixels
    if np.any(valid):
        mean_difference = float(difference[valid].mean())
    else:
```

```
        mean_difference = float("nan")

    return (
        matches_img, aligned, difference,
        len(good), int(inliers.sum()), valid,
        mean_difference
    )
```

```
Template: C:\EN3160 - Image Processing and Machine Vision\a-02\a2images\a2images\pcbs\AECC1.jpg
PCB images: ['AAAA1.jpg', 'ADAD1.jpg', 'AECC2.jpg', 'AECC3.jpg', 'AECC4.jpg']
```

In [149…

```
results = []

for path in image_files:
    print(f"\nProcessing {path.name}...")

    pcb = cv2.imread(str(path))

    if pcb is None:
        print("Skipping unreadable image.")
        continue

    try:
        (
            matches_img, aligned, difference,
            n_good, n_inliers, valid, mean_difference
        ) = align_and_compare(template, pcb)

    except (ValueError, cv2.error) as error:
        print(f"Skipping {path.name}: {error}")
        continue

    # Save outputs
    name = path.stem

    cv2.imwrite(
        str(OUTPUT_DIR / f"{name}_matches.jpg"),
        matches_img
    )
    cv2.imwrite(
        str(OUTPUT_DIR / f"{name}_aligned.jpg"),
        aligned
    )
    cv2.imwrite(
        str(OUTPUT_DIR / f"{name}_difference.png"),
        difference
    )

    # Display results
    fig, axes = plt.subplots(1, 4, figsize=(18, 5))

    axes[0].imshow(cv2.cvtColor(matches_img, cv2.COLOR_BGR2RGB))
    axes[0].set_title(f"{path.name}: RANSAC matches")

    axes[1].imshow(cv2.cvtColor(aligned, cv2.COLOR_BGR2RGB))
    axes[1].set_title("Aligned PCB")

    axes[2].imshow(difference, cmap="gray", vmin=0, vmax=255)
    axes[2].set_title("Absolute difference")

    # Hide invalid warp regions for a clearer comparison
    difference_valid = np.ma.masked_where(~valid, difference)
    axes[3].imshow(difference_valid, cmap="gray", vmin=0, vmax=255)
    axes[3].set_title("Difference (valid region)")

    for ax in axes:
        ax.axis("off")

    fig.suptitle(
        f"{path.name} | Good matches: {n_good} | "
        f"RANSAC inliers: {n_inliers} | "
        f"Mean difference: {mean_difference:.2f}"
    )

    plt.tight_layout()
    plt.show()

    results.append({
        "Image": path.name,
        "Good matches": n_good,
        "RANSAC inliers": n_inliers,
```

```
        "Mean difference": mean_difference
    })

print("\nSaved results to:", OUTPUT_DIR.resolve())

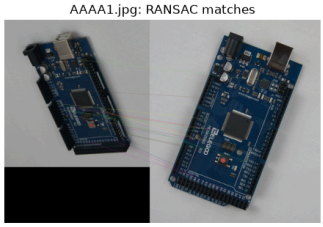
# Summary table
import pandas as pd

results_df = pd.DataFrame(results)
display(results_df.round(2))

results_df.to_csv(
    OUTPUT_DIR / "q4_summary.csv",
    index=False
)
```

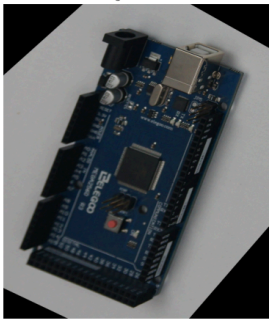
```
Processing AAAA1.jpg...
```

AAAA1.jpg: RANSAC matches

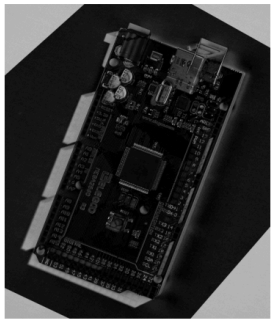


AAAA1.jpg | Good matches: 51 | RANSAC inliers: 18 | Mean difference: 28.29

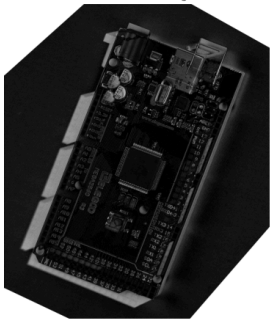
Aligned PCB



Absolute difference

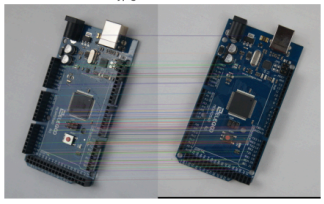


Difference (valid region)



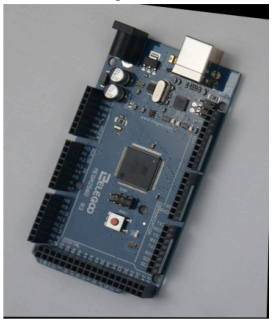
```
Processing ADAD1.jpg...
```

ADAD1.jpg: RANSAC matches

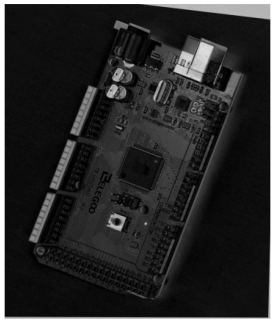


ADAD1.jpg | Good matches: 369 | RANSAC inliers: 194 | Mean difference: 31.16

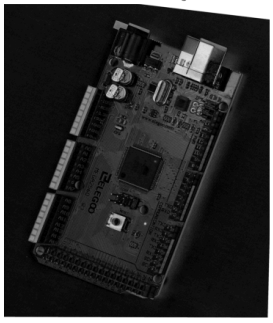
Aligned PCB



Absolute difference

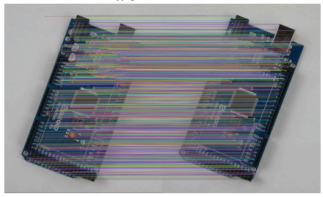


Difference (valid region)



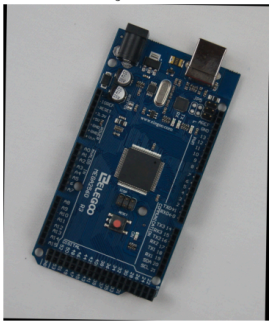
```
Processing AECC2.jpg...
```

AECC2.jpg: RANSAC matches

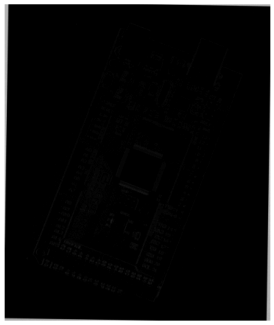


AECC2.jpg | Good matches: 4322 | RANSAC inliers: 3847 | Mean difference: 3.03

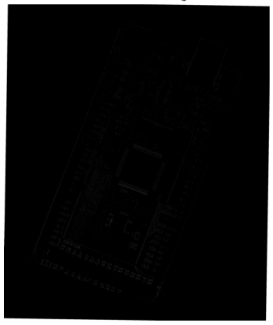
Aligned PCB



Absolute difference



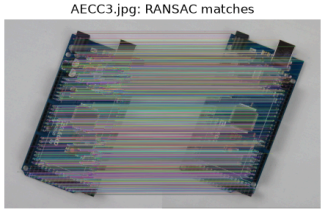
Difference (valid region)



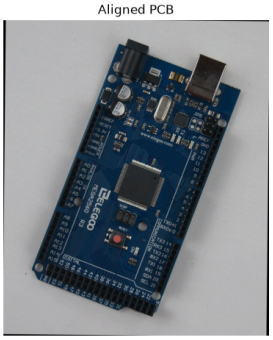
```
Processing AECC3.jpg...
```

AECC3.jpg | Good matches: 5629 | RANSAC inliers: 5006 | Mean difference: 2.81

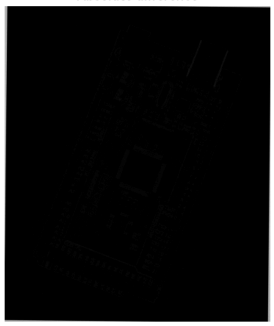
AECC3.jpg: RANSAC matches



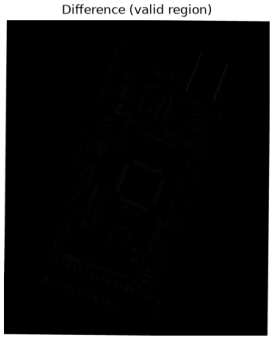
Aligned PCB



Absolute difference



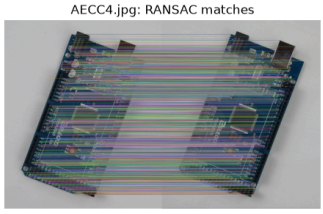
Difference (valid region)



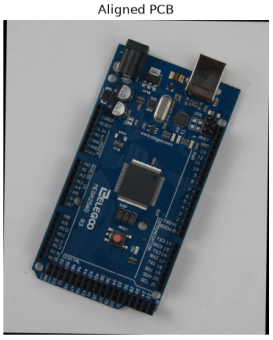
```
Processing AECC4.jpg...
```

AECC4.jpg | Good matches: 3830 | RANSAC inliers: 3402 | Mean difference: 3.61

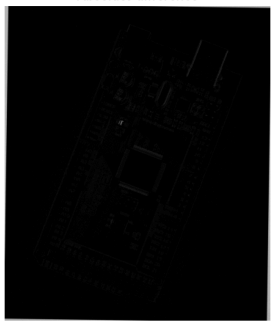
AECC4.jpg: RANSAC matches



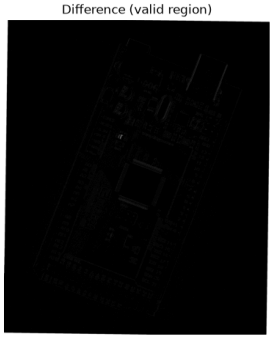
Aligned PCB



Absolute difference



Difference (valid region)



```
Saved results to: C:\EN3160 - Image Processing and Machine Vision\a-02\q4_outputs
```

	Image	Good matches	RANSAC inliers	Mean difference
0	AAAA1.jpg	51	18	28.29
1	ADAD1.jpg	369	194	31.16
2	AECC2.jpg	4322	3847	3.03
3	AECC3.jpg	5629	5006	2.81
4	AECC4.jpg	3830	3402	3.61